# Treinador de Expressões Regulares

Guilherme Piacentini da Silva[1], Herton da Silveira e Silva[1]

[1]Centro de Ciências Tecnológicas (CCT)
Universidade do Estado de Santa Catarina (UDESC), Joinville, SC, Brasil

{gui.piacentini, herton.silveira}@edu.udesc.br

**Resumo.** [depois]

## 1. Introdução

[depois]

## 2. Entrega da 1ª semana: Tema e Público-Alvo

O tema deste projeto é o desenvolvimento de um protótipo de ferramenta web para a prática de expressões regulares (regex). A ferramenta apresentará ao usuário uma série de desafios, e cada desafio terá um conjunto de exemplos que devem ser aceitos e outro conjunto que deve ser rejeitado. O aluno escreve uma expressão regular e a ferramenta a testa automaticamente contra esses exemplos, indicando quais casos passaram e quais falharam. O objetivo pedagógico é oferecer prática guiada e com correção imediata.

A aplicação deve rodar inteiramente no navegador, usa o próprio motor de expressões regulares do JavaScript para fazer a verificação e trabalha com um conjunto fixo de desafios de dificuldade crescente.

O público-alvo são estudantes de graduação em Ciência da Computação e cursos parecidos, em especial os que estão tendo o primeiro contato com expressões regulares, seja em disciplinas introdutórias de programação, seja em disciplinas de linguagens formais. As expressões regulares são muito usadas na busca e na validação de texto, mas costumam ser difíceis para iniciantes por causa da sintaxe diferente. A ferramenta pretende atender o aluno que já entende a ideia geral, mas que precisa de um ambiente simples para praticar e ganhar confiança antes de aplicar regex em trabalhos reais.

## 3. Entrega da 2ª semana: Benchmark e Trabalhos Correlatos

Já existem várias ferramentas on-line voltadas ao aprendizado e à prática de expressões regulares. Para o benchmark, elas foram agrupadas em três tipos: testadores, tutoriais guiados e jogos. Além disso, foi considerada uma ferramenta acadêmica da área de linguagens formais.

- **Testadores (Regex101 e RegExr):** permitem escrever uma expressão e ver em tempo real o que ela casa em um texto, com explicação da expressão e folha de referência.
- **Tutoriais guiados (RegexOne e RegexLearn):** o RegexOne traz lições interativas com exercícios progressivos; o RegexLearn oferece um curso passo a passo, um playground e uma seção de prática, com suporte a vários idiomas, incluindo o português.
- **Jogo (Regex Crossword):** usa palavras cruzadas em que o jogador preenche uma grade satisfazendo expressões regulares.

- **Software acadêmico (JFLAP):** ferramenta de linguagens formais e autômatos que permite construir autômatos e expressões.

A Tabela 1 resume a comparação entre essas ferramentas e a proposta deste projeto.

Tabela 1: Comparação entre ferramentas existentes e a proposta deste projeto.

| **Ferramenta** | **Tipo / modelo** | **Dificuldade progressiva** | **Em português** | **Modelo aceitar/rejeitar** |
|---|---|---|---|---|
| Nossa proposta | Prática por desafios (aceitar/rejeitar) | Sim | Sim | Sim |
| Regex101 | Testador e depurador | Não | Não | Não |
| RegExr | Testador com referência | Não | Não | Não |
| RegexOne | Tutorial guiado | Sim | Não | Não |
| RegexLearn | Tutorial e playground | Sim | Sim | Não |
| RegexCross | Jogo (palavras cruzadas) | Sim | Não | Não |
| JFLAP | Software acadêmico (autômatos) | Não | Não | Parcial |

A análise mostra que já existem boas ferramentas, e a nossa proposta não pretende substituí-las, mas ocupar um nicho específico. O Regex101 e o RegExr focam em testar e depurar, não em exercitar com correção automática. Os tutoriais ensinam bem, porém misturam teoria e prática e nem sempre estão em português. O Regex Crossword é divertido, mas o modelo de cruzadas se distancia do uso cotidiano de regex. O JFLAP é acadêmico e de desktop.

A nossa ferramenta se diferencia por ser simples, em português, rodar no navegador e se concentrar em um único modelo de prática: o aluno escreve uma expressão e ela é validada automaticamente contra listas de exemplos que devem ser aceitos e rejeitados. Esse formato de aceitar e rejeitar aproxima a prática do raciocínio de linguagens formais e dá um retorno objetivo ao aluno.

No levantamento de trabalhos correlatos, foram selecionados dois artigos científicos:

- **Automata Tutor v3 (D'Antoni et al., 2020):** os autores desenvolveram uma ferramenta web que corrige automaticamente e fornece feedback personalizado em exercícios de linguagens formais, incluindo expressões regulares, autômatos e conversões entre representações. A proposta parte de técnicas que comparam a resposta do aluno com a solução esperada e geram dicas sobre o erro. Como resultados, a ferramenta foi usada por milhares de estudantes em dezenas de países e, em uma turma, uma pesquisa com cerca de 950 alunos indicou avaliação favorável quanto à usabilidade e à utilidade.

- **EduLing (Dognini e Raabe, 2003):** os autores desenvolveram um software educacional para linguagens regulares, com uma interface gráfica interativa em que o aluno constrói autômatos e expressões regulares, alterna entre as diferentes representações e simula o reconhecimento de sentenças, além de aplicar algoritmos de conversão para mostrar a equivalência entre elas. Após a experimentação, os alunos responderam a um questionário, e os resultados preliminares indicaram que a ferramenta pode contribuir para a qualidade do ensino de Computação.

# 4. Entrega da 3ª semana: Concepção do Produto

Nesta fase é descrita a concepção do produto, com os casos de uso preliminares, um protótipo de baixa fidelidade e o planejamento inicial da interação com o usuário.

## 4.1 Casos de uso (preliminar)

A ferramenta tem um único ator, o Aluno, que interage com o sistema pelo navegador. Os casos de uso preliminares identificados são:

- **UC01 — Visualizar desafio:** ler o enunciado e as listas de exemplos que devem ser aceitos e rejeitados.
- **UC02 — Submeter expressão regular:** digitar uma expressão e solicitar o teste.
- **UC03 — Visualizar resultado:** ver quais exemplos passaram e quais falharam.
- **UC04 — Pedir dica:** consultar uma dica sobre o desafio atual.
- **UC05 — Navegar entre desafios:** avançar para o próximo ou voltar ao anterior.
- **UC06 — Acompanhar progresso:** ver quantos desafios já foram concluídos.

## 4.2 Protótipo de baixa fidelidade (preliminar)

A Figura 1 apresenta o wireframe da tela principal. O topo mostra o título e o progresso. Logo abaixo fica o enunciado do desafio e, em seguida, duas colunas com os exemplos que devem ser aceitos e rejeitados. O aluno digita a expressão no campo central e aciona o botão Testar. A área de resultado indica, de forma visual, quais exemplos passaram e quais falharam. No rodapé ficam o atalho para a dica e os botões de navegação.

## 4.3 Planejamento de interação (preliminar)

A interação segue um ciclo simples de tentativa e correção:

1. O aluno visualiza o desafio, com o enunciado e as listas de exemplos que devem ser aceitos e rejeitados.
2. Ele digita uma expressão regular e aciona o botão Testar.
3. O sistema dá um retorno imediato e visual, marcando cada exemplo como correto ou incorreto.
4. Enquanto houver exemplos incorretos, o aluno ajusta a expressão e testa de novo, podendo também pedir uma dica.
5. Quando todos os exemplos ficam corretos, o desafio é concluído e o aluno pode avançar para o próximo.

# 5. Entrega da 4ª semana: Requisitos e Tecnologias

Nesta fase são apresentados os requisitos funcionais e não-funcionais do produto e a proposta preliminar de tecnologias a serem utilizadas.

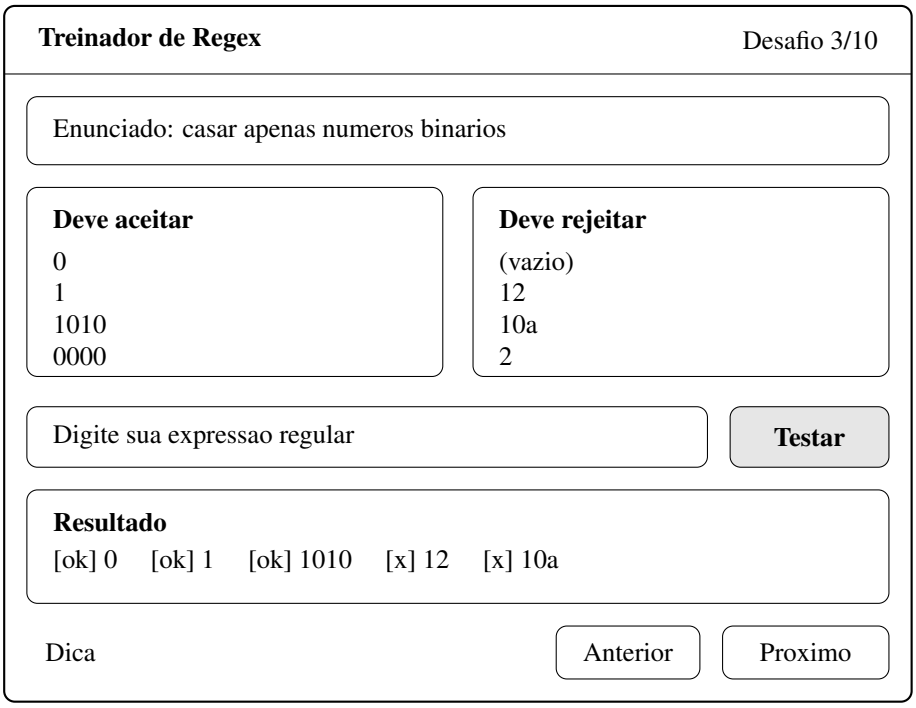



Figura 1: Protótipo de baixa fidelidade (wireframe) da tela principal.

### 5.1 Requisitos funcionais

Os requisitos funcionais descrevem o que o sistema deve fazer (DENNIS; WIXOM; ROTH, 2014). Para a ferramenta, foram definidos:

- **RF01:** exibir um desafio por vez, com o enunciado do que a expressão precisa casar.
- **RF02:** mostrar a lista de exemplos que devem ser aceitos e a que deve ser rejeitada.
- **RF03:** oferecer um campo para o usuário digitar a expressão regular.
- **RF04:** ao acionar o botão Testar, aplicar a expressão a todos os exemplos e marcar cada um como correto ou incorreto.
- **RF05:** sinalizar o resultado de forma visual, diferenciando acertos e erros.
- **RF06:** marcar o desafio como concluído quando todos os exemplos a aceitar passam e todos os a rejeitar são barrados.
- **RF07:** permitir avançar para o próximo desafio e voltar ao anterior.
- **RF08:** exibir o progresso do aluno, como o número de desafios concluídos.
- **RF09:** disponibilizar uma dica por desafio.
- **RF10:** tratar expressões inválidas sem travar, exibindo uma mensagem de erro.

### 5.2 Requisitos não-funcionais

Requisitos não-funcionais descrevem restrições e qualidades do sistema, como desempenho, usabilidade e portabilidade, e não funcionalidades específicas (WAZLAWICK, 2015, p. 57-61). Para a ferramenta, foram definidos:

- **RNF01 (Arquitetura):** a aplicação deve rodar inteiramente no navegador, sem servidor nem banco de dados.
- **RNF02 (Implementação):** a verificação deve usar o motor de expressões regulares nativo do JavaScript.
- **RNF03 (Usabilidade):** a interface deve ser simples e intuitiva para iniciantes.
- **RNF04 (Portabilidade):** deve funcionar nos principais navegadores e se adaptar a telas de computador e de celular.
- **RNF05 (Desempenho):** o retorno após o teste deve ser praticamente imediato.
- **RNF06 (Manutenibilidade):** os desafios devem ficar em um arquivo de dados fixo e separado do código, fácil de editar e de ampliar.

### 5.3 Proposta preliminar de tecnologias

A proposta inicial de tecnologias é descrita a seguir:

- **HTML5:** linguagem de marcação que estrutura as páginas e os elementos da interface, como o campo de entrada, os botões e as áreas de texto.
- **CSS3:** linguagem de estilo responsável pela aparência e pelo layout responsivo, para que a ferramenta funcione bem no computador e no celular.
- **JavaScript:** linguagem que implementa a lógica da aplicação e faz a verificação das expressões por meio do objeto nativo RegExp, sem depender de bibliotecas externas.
- **JSON:** formato usado para armazenar o conjunto fixo de desafios (enunciado, exemplos a aceitar e a rejeitar e dica), separando os dados do código e facilitando a edição e a ampliação do banco.
- **Git e GitHub:** ferramentas de controle de versão e de hospedagem gratuita, via GitHub Pages, já que o produto é totalmente estático.
- **Figma:** software de criação gráfica que será usado para evoluir o wireframe atual até um protótipo de alta fidelidade nas próximas entregas.
- **Inteligência Artificial:** a ferramenta não usa IA em seu funcionamento, pois a verificação é determinística. Durante o desenvolvimento, porém, assistentes de IA podem apoiar a geração de exemplos para os desafios e a revisão do código.

Como o escopo é pequeno, optou-se por não adotar frameworks pesados. A aplicação pode ser construída apenas com HTML, CSS e JavaScript, o que reduz a complexidade e facilita a hospedagem.

## Referências


D'ANTONI, L.; HELFRICH, M.; KRETINSKY, J.; RAMNEANTU, E.; WEININGER, M. Automata Tutor v3. In: International Conference on Computer Aided Verification (CAV). Lecture Notes in Computer Science, v. 12225, p. 3–14, 2020.

DENNIS, A.; WIXOM, B. H.; ROTH, R. M. **Análise e projeto de sistemas**. 5. ed. Rio de Janeiro: LTC, 2014.

DOGNINI, M. J.; RAABE, A. L. A. EduLing: software educacional para linguagens regulares. In: XIV Simpósio Brasileiro de Informática na Educação (SBIE), 2003, p. 216–225.

LEDUR, C. L. **Análise e projeto de sistemas**. Porto Alegre: SAGAH, 2017.

REGEX101. **Online regex tester and debugger**. Disponível em: https://regex101.com/. Acesso em: 6 out. 2026.

REGEX CROSSWORD. **A crossword puzzle game using regular expressions**. Disponível em: https://regexcrossword.com/. Acesso em: 6 out. 2026.

REGEXLEARN. **Step by step, interactive regex tutorial**. Disponível em: https://regexlearn.com/. Acesso em: 6 out. 2026.

REGEXONE. **Learn Regular Expressions with simple, interactive exercises**. Disponível em: https://regexone.com/. Acesso em: 6 out. 2026.

REGEXR. **Learn, build and test Regex**. Disponível em: https://regexr.com/. Acesso em: 6 out. 2026.

RODGER, S. H.; FINLEY, T. W. **JFLAP: an interactive formal languages and automata package**. Sudbury: Jones and Bartlett Publishers, 2006.

WAZLAWICK, R. S. **Análise e design orientados a objetos para sistemas de informação: modelagem com UML, OCL e IFML**. 3. ed. Rio de Janeiro: Elsevier, 2015.